

LIBRARY MANAGEMENT SYSTEM

A Java and MySQL Based Console Application

MINI PROJECT REPORT

PROJECT SUBMISSION

Library Management System

PREPARED BY

Utkarsh Dixit

ROLL NO. 2500270130204

BRANCH Information Technology

SEMESTER 3rd

ACADEMIC SESSION 2026



DECLARATION

I hereby declare that the project report entitled “Library Management System” is prepared for academic submission. The project has been developed to demonstrate programming and database concepts through a working Java application connected with a MySQL database.

The screenshots included in this report represent the project source code, console execution, and database output used during development and testing.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my teachers and department for providing the opportunity to develop this mini project. Their academic guidance helped me understand how Java programming concepts can be integrated with a relational database to create a practical application.

I also acknowledge the usefulness of IntelliJ IDEA, Java, JDBC, and MySQL Workbench during development, debugging, database verification, and testing of the Library Management System.

ABSTRACT

The Library Management System is a console-based Java application designed to simplify common library operations such as adding books, viewing available books, registering users, borrowing books, and returning books. The application uses JDBC to connect Java code with a MySQL database so that information can be stored and retrieved persistently.

The project follows a modular structure in which the main program handles user interaction while DAO classes perform database operations. Separate entities and database tables are used for books, users, and borrowed-book records. This separation makes the application easier to understand, maintain, test, and extend.

The supplied execution evidence shows the application menu running successfully in IntelliJ IDEA and the MySQL Workbench result grid displaying book records in the library database. These results provide practical evidence that the application and database components are working together.

TABLE OF CONTENTS

1. Introduction

Introduces the project background, purpose, and the need for a computerized system to manage library records.

2. Objectives

Describes the functional and technical goals of the project, including book, user, borrowing, returning, and database management.

3. Technologies Used

Explains Java, JDBC, MySQL, MySQL Workbench, IntelliJ IDEA, and Maven and their roles in the implementation.

4. System Requirements

Lists the software and hardware environment needed to develop, run, and test the application.

5. System Design and Architecture

Explains the overall flow between the console interface, Java classes, DAO layer, JDBC connection, and MySQL database.

6. Database Design

Describes the database, tables, important fields, relationships, and how records are stored for books, users, and borrowing activity.

7. Modules of the System

Provides a detailed explanation of each major feature and the operations performed by the corresponding Java components.

8. Implementation

Describes the Java classes, JDBC connectivity, SQL operations, validation, and menu-driven program flow.

9. Testing and Results

Documents functional testing and uses the submitted IntelliJ and MySQL Workbench screenshots as execution evidence.

10. Advantages and Limitations

Explains the practical benefits of the current system and the limitations of a basic console-based implementation.

11. Future Enhancements

Suggests improvements such as GUI, authentication, reports, fine calculation, and stronger validation.

12. Conclusion

Summarizes what was achieved and the programming/database concepts demonstrated by the project.

13. References

Lists the technical references and tools used for understanding and implementing the project.

1. INTRODUCTION

The Library Management System is designed to computerize routine library activities that otherwise require manual record keeping. In a manual system, information about books, users, and issued books may be maintained in registers or separate files, making searching and updating slower and more error-prone. A database-backed application provides a structured way to store and retrieve these records.

The developed project uses Java as the application language and MySQL as the relational database. JDBC acts as the communication layer between the Java program and MySQL. The console interface gives the operator a simple menu through which major operations can be selected.

The system is intentionally kept suitable for a mini-project environment. Its modular organization demonstrates how application logic and database operations can be separated using DAO classes, while the database tables provide persistent storage for the project data.

2. OBJECTIVES

The primary objective is to build a simple working application that demonstrates the integration of Java programming with a relational database. The project focuses on implementing practical operations rather than only demonstrating isolated code fragments.

Major objectives include adding new book records, displaying stored books, adding and viewing users, borrowing a book, returning a book, and maintaining the availability state of books. Another objective is to understand JDBC connection handling and SQL statements from Java.

The project also aims to provide a clear modular structure. By keeping database operations inside DAO classes and maintaining separate data classes, the code can be modified or expanded without redesigning the complete application.

3. TECHNOLOGIES USED

Java is used for the application logic, console interaction, classes, objects, methods, and control flow. The project uses standard Java utilities such as Scanner for reading user input.

JDBC (Java Database Connectivity) is used to establish communication between Java and MySQL. Through JDBC, the program can execute SQL statements and retrieve results from database tables.

MySQL stores persistent project data, while MySQL Workbench is used to create, inspect, and verify the database. IntelliJ IDEA is used as the development environment, and Maven manages the project structure and dependencies.

4. SYSTEM REQUIREMENTS

Hardware requirements are modest because the project is a console application. A normal personal computer with sufficient RAM and storage for Java, IntelliJ IDEA, Maven, and MySQL is adequate for development and testing.

Software requirements include a compatible JDK, IntelliJ IDEA, MySQL Server, MySQL Workbench, and the JDBC driver dependency. The project should be configured with the correct database URL, username, password, and JDBC driver.

A working local MySQL server is required when the application performs database operations. Internet access is useful for downloading tools or Maven dependencies during setup, but normal local execution of the completed application can work without continuous internet access.

5. SYSTEM DESIGN AND ARCHITECTURE

The application follows a simple layered design. The Main class provides the console menu and collects user input. Data/entity classes represent records such as books and users. DAO classes contain SQL operations and communicate with MySQL through JDBC.

The typical flow is: user selects an operation → Main validates/collects input → the relevant DAO method is called → JDBC sends SQL to MySQL → the database returns a result → the application displays the result to the user. This approach keeps user interaction separate from database-specific code.

The architecture is appropriate for a mini project because it is easy to trace during debugging and provides a foundation for later additions such as a GUI or web interface.

6. DATABASE DESIGN

The project uses a MySQL database named `library_management`. The database stores information in separate tables so that different types of records can be maintained in a structured manner.

The `books` table contains book-related information such as `book_id`, `title`, `author`, and `availability`. The `users` table is intended for library-user records, while `borrowed_books` records borrowing-related information. Separating these tables reduces duplication and makes the database easier to query.

The submitted MySQL Workbench evidence shows the library database being queried successfully. The displayed `books` table contains two records: “Java Programming” by James Gosling and “Python programming” by Guido van Rossum, with `availability` values shown as 1. This directly demonstrates that records are present in the database.

7. MODULES OF THE SYSTEM

Add Book: accepts book details and inserts a new record into the books table. This module demonstrates data insertion through JDBC.

View Books: retrieves stored book records and displays them to the user. It is useful for checking the current inventory and availability state.

Add User and View Users: manage basic library-user records. These functions demonstrate the same database CRUD pattern for a different entity.

Borrow Book: records a borrowing action and changes the relevant book availability so that the same book is not treated as freely available while borrowed.

Return Book: processes the return operation and updates the book availability. Together, borrowing and returning demonstrate a simple transaction-oriented workflow.

8. IMPLEMENTATION

The Java project is organized under the `com.library` package. The `Main` class contains the menu-driven entry point, while classes such as `Book`, `BookDAO`, `UserDAO`, and `BorrowDAO` represent data and database operations. A database connection class provides the JDBC connection used by DAO methods.

Prepared SQL statements or standard JDBC statements can be used to perform `INSERT`, `SELECT`, `UPDATE`, and related operations. `ResultSet` is used when rows are retrieved from MySQL. Proper connection and statement handling is important for reliable execution.

The submitted IntelliJ evidence shows the project source tree, `Main.java`, multiple DAO/entity classes, and the running console. The console displays the `LIBRARY MANAGEMENT SYSTEM` menu with options from `Add Book` through `Exit`, and the run terminates with exit code 0 after selecting option 7.

9. TESTING AND RESULTS

Testing was performed by running the Java application and checking database results in MySQL Workbench. Functional testing focuses on whether the menu starts correctly, database records can be viewed, and the application can terminate normally.

The IntelliJ screenshot provides evidence of successful application execution. It shows the Library Management System menu with seven options and the message “Thank you for using Library Management System!” followed by “Process finished with exit code 0”.

The MySQL Workbench screenshot provides database evidence. The Result Grid shows two rows in the books table and the Action Output records successful SELECT operations. Together, these screenshots demonstrate both the application layer and database layer of the project.

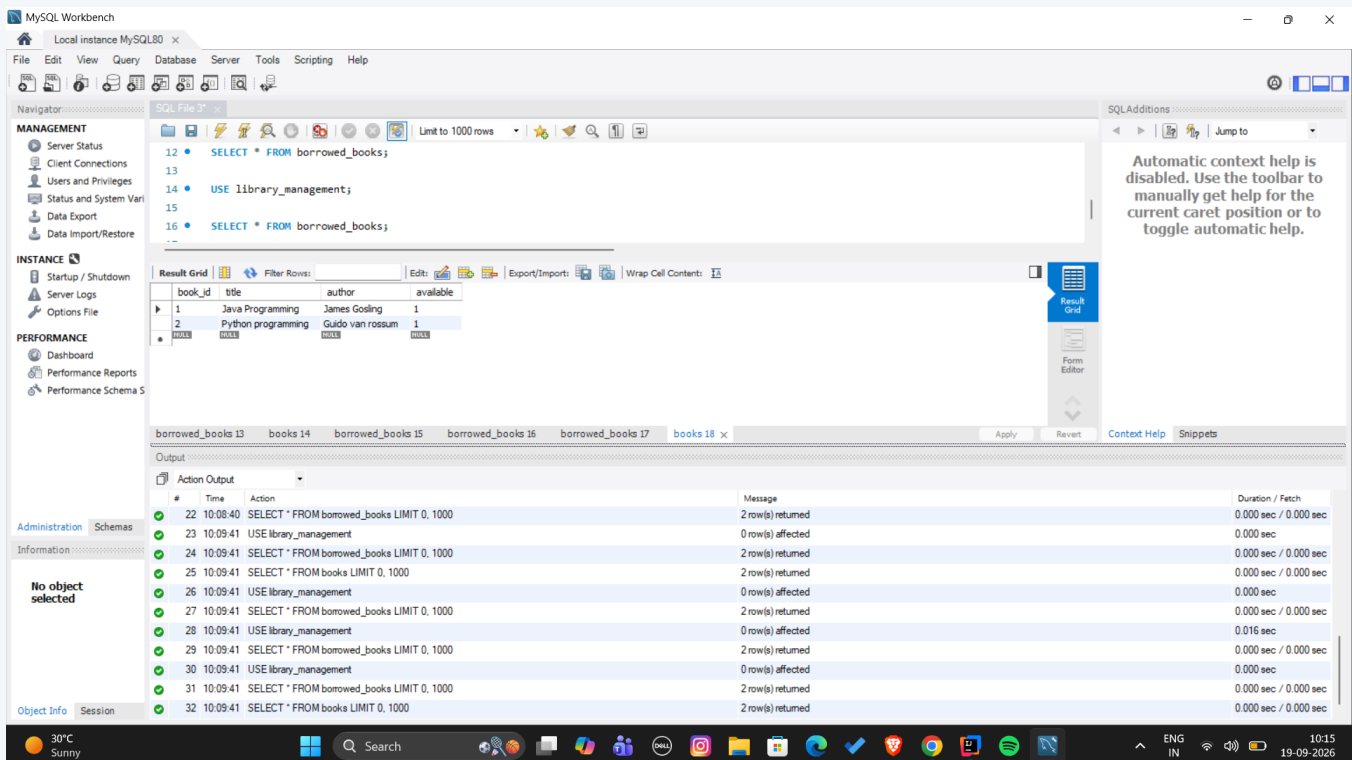


Figure 1: MySQL Workbench database evidence – books table and query output

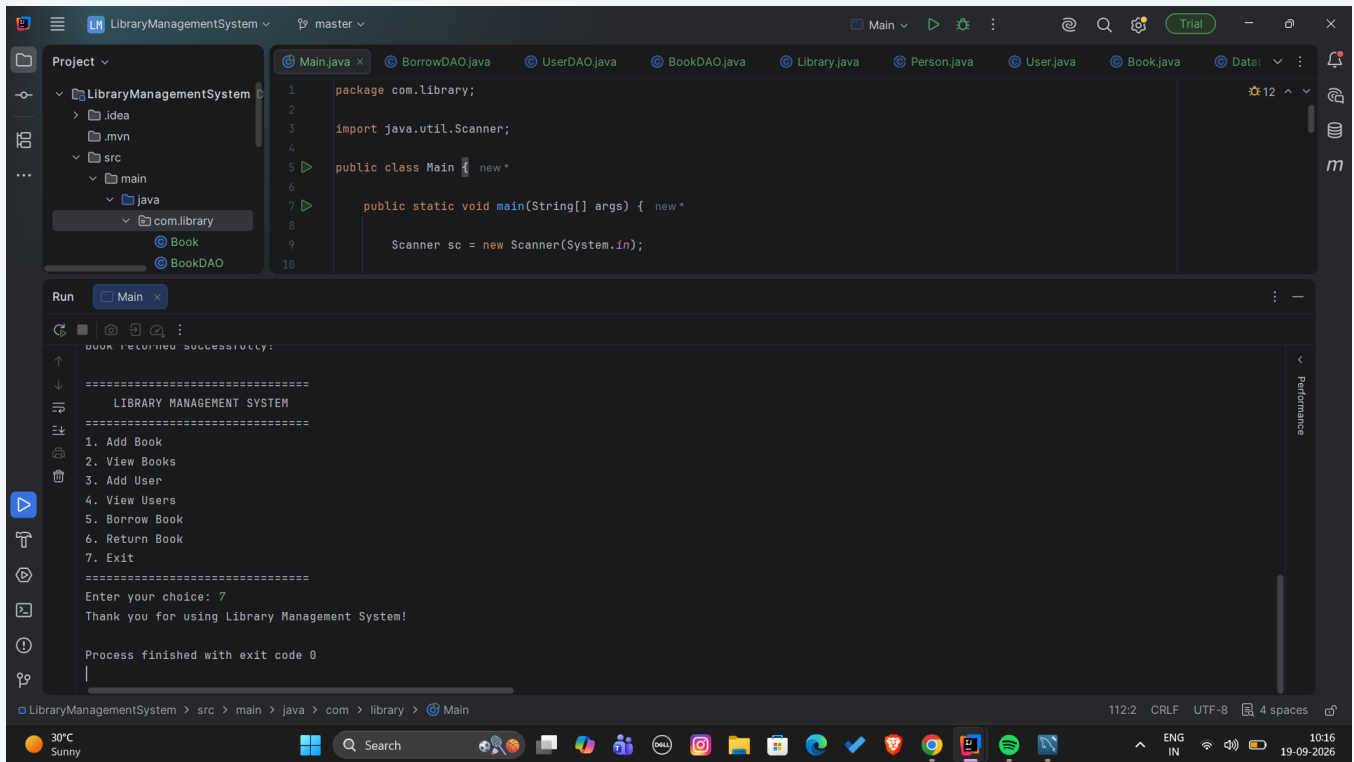


Figure 2: IntelliJ IDEA evidence – project structure, source code and successful console execution

The figures above are the replacement evidence supplied for the final report. They are included directly in the Testing and Results section so that the documented results are supported by the actual project screenshots.

10. ADVANTAGES AND LIMITATIONS

Advantages include simple operation, persistent MySQL storage, modular Java code, easy database verification, and a clear menu-driven workflow. The DAO structure also makes database logic easier to organize.

Because the application is console-based, it is lightweight and easy to run during a classroom demonstration. The database provides more reliable persistent storage than keeping all records only in memory.

Limitations include the absence of a graphical interface, advanced authentication, fine calculation, search filters, detailed reports, and extensive input validation. The system is primarily designed as an academic mini project rather than a full commercial library platform.

11. FUTURE ENHANCEMENTS

The system can be extended with a Java Swing or JavaFX graphical interface to make it easier for non-technical users. A web-based interface could also be created using a Java web framework.

Additional database features could include member login, librarian authentication, due dates, automatic fine calculation, search by title/author, category management, reservation of books, and borrowing history reports.

Other improvements could include stronger validation, exception handling, database transactions, logging, backup facilities, and role-based access control. These additions would make the system more suitable for a larger real-world deployment.

12. CONCLUSION

The Library Management System successfully demonstrates the integration of Java, JDBC, and MySQL in a practical mini-project. It covers common library operations through a simple console menu and stores persistent records in a relational database.

The supplied execution evidence confirms that the application menu runs successfully in IntelliJ IDEA and that the MySQL database contains book records that can be retrieved through SQL queries. The project therefore provides practical experience in Java programming, database connectivity, SQL, and modular application design.

Overall, the project establishes a useful foundation for further development into a GUI-based or web-based library management application.

13. REFERENCES

Java Platform documentation and standard Java library concepts.

MySQL documentation and SQL reference concepts.

JDBC concepts for Java-to-database connectivity.

IntelliJ IDEA documentation and project development workflow.

Maven project structure and dependency-management concepts.